Habit Tycoon

Bug Remediation Plan

Implementation plan for five identified security, completion, streak, and dividend-accounting defects.

Priority	Workstream	Outcome
P0	Stock RPC authorization	Only the signed-in user can trade their own account.
P1	Multi-goal completion	Goals above one work without duplicate failures.
P1	Schedules and streaks	Rest days are enforced and streaks continue correctly.
P1	Dividend undo accounting	Undo has no stranded payouts or partial state.

Recommended delivery sequence

- Immediately disable or patch the unsafe RPC entry points before feature work. Database authorization must not depend on the Angular client passing the correct UUID.

- Ship the multi-goal database migration and rest-day guard together. Both affect completion eligibility and should be tested as one release.

- Repair streak calculation next, then move completion and undo accounting into database RPCs so every money-changing operation is atomic.

- Keep undo disabled for dividend-bearing completions until the accounting RPC has been deployed and reconciled in a staging database.

01. Authorize stock-trading RPCs from the session

P0 security - purchase_stock_shares and sell_stock_shares are SECURITY DEFINER functions but trust buyer_id/seller_id supplied by the caller. A signed-in attacker can submit another account's UUID.

Implementation plan

- Create a forward-only SQL migration that changes both RPCs to derive the actor as auth.uid(); remove buyer_id and seller_id from their public signatures. Return an authentication error when auth.uid() is null.

- Use a row lock (SELECT ... FOR UPDATE) on business_stocks and the caller's user_profiles row before checking available shares and cash. This prevents two concurrent purchases from overselling shares or overspending cash.

- Update Angular wrappers to call purchase_stock_shares(stock_uuid, shares_to_buy) and sell_stock_shares(stock_uuid, shares_to_sell). Do not expose user IDs as authorization inputs.

- Audit every other authenticated SECURITY DEFINER function. At minimum, create_business_stock, send_habit_poke, send_stockholder_reminder, and dividend functions must either use auth.uid() or validate the caller's relationship to the target record.

- Set an explicit safe search_path inside each SECURITY DEFINER function (for example, SET search_path = public, pg_temp) to reduce object-shadowing risk.

Verification

- Integration test: user A can buy and sell their own holdings.

- Negative test: user A invokes the RPC with user B's former UUID argument or attempts the old signature; it must fail.

- Concurrency test: two purchases competing for the last shares produce one success and one clean failure, with no negative availability or negative cash.

Rollback / operational notes

- Deploy as an additive migration: create secure replacement functions, update grants, deploy client, then revoke or drop the old signatures.

- Before deployment, identify and update any external clients that call the old signatures. Keep a database backup and record balances/holdings totals for reconciliation.

Primary touchpoints: stocks-system-functions.sql; src/app/services/habit-business.service.ts; src/app/stocks/stocks.page.ts

02. Make multi-completion goals compatible with the database

P1 functional - The application inserts a habit_completions row for every increment, while the current unique index permits only one row per habit/user/day. Goals above one fail on their second completion.

Implementation plan

- Decide the invariant explicitly: multiple completion events per habit and local calendar day are valid up to goal_value. Keep an event row for each completion so earnings and audit history remain accurate.

- Create a migration that drops idx_unique_daily_completion. Do not replace it with another one-row-per-day unique index.

- Move the eligibility check and increment into a database completion RPC. The RPC should lock the habit row, count completions within the current period, reject counts at or above goal_value, insert one event, and update balances/stats in the same transaction.

- Use the same period definition in the RPC and client. Prefer an explicit user timezone stored in the profile or a clear UTC policy; do not combine local-day client timestamps with UTC-only database indexes.

- Add a narrow index for period queries, such as (habit_business_id, user_id, completed_at DESC), after verifying query plans.

Verification

- A goal of three accepts exactly three same-period completions and rejects a fourth.

- Two rapid completion requests do not exceed goal_value.

- A next-period completion succeeds after reset, while prior event history and earnings remain intact.

Rollback / operational notes

- Take a schema backup before dropping the index. The change is safe for existing event history, but verify no external process assumes one row per day.

- If the new RPC rollout must be staged, temporarily hide goals greater than one rather than leaving a broken UI path available.

Primary touchpoints: 001-add-unique-completion-constraint.sql; src/app/services/habit-business.service.ts; habit_completions schema

03. Enforce specific-day schedules at both UI and service layers

P1 functional - The home screen displays a Complete control on rest days and completeHabit does not reject a specific_days habit when today is not active.

Implementation plan

- In the template, hide or disable the completion interaction when !isTodayActiveDay(hb), retaining the existing Next label and rest-day status.

- Add a service-level guard immediately after resolving the interval: if it is specific_days and isTodayActiveDay is false, return a clear error before creating a completion or changing money.

- Enforce the identical condition in the new database completion RPC. Client checks improve UX; database checks protect every client and direct API call.

- Validate active_days on create and edit: values must be integers from 0 through 6, non-empty for specific_days, and deduplicated.

Verification

- A Monday/Wednesday habit has no enabled Complete action on Tuesday and direct service/RPC attempts are rejected.

- The action is enabled at the user's configured day boundary on an active day.

- Malformed schedules are refused during creation and editing.

Rollback / operational notes

- This is backward-compatible. Existing legacy weekly rows should be migrated to all seven days before enabling the database guard.

- Monitor rejected completion attempts for one release to catch timezone or schedule-migration mistakes.

Primary touchpoints: src/app/home/home.page.html; src/app/services/habit-business.service.ts; src/app/services/habit-interval.service.ts; 010-specific-days-habit.sql

04. Calculate streaks from completed periods, not partial events

P1 functional - For a multi-goal habit, a partial completion updates last_completed_at to today. When the final completion runs, streak logic sees today's partial event instead of the prior completed period and resets the streak to one.

Implementation plan

- Separate activity tracking from successful-period tracking. Either add last_goal_completed_at or derive the prior completed period from completion events whose count reached goal_value. Do not overwrite the streak anchor on partial completion.

- On the final event of a period, calculate whether the immediately preceding eligible period was fully completed, then increment or reset the streak exactly once.

- For specific_days, determine the preceding eligible day from active_days rather than simply subtracting 24 hours.

- Make the completion RPC return the authoritative progress, streak, earnings, and next-reset information; reload or update the client from that response.

- Correct existing streaks only after deciding the historical policy. A one-time reconciliation can recompute from habit_completions if event history is trustworthy.

Verification

- A two-goal habit completed on two consecutive eligible days reaches streak two after each day is fully completed.

- A partial day followed by a missed eligible day resets the streak.

- A Monday/Wednesday schedule carries a streak Monday to Wednesday, but not Monday to the following Monday if Wednesday was missed.

Rollback / operational notes

- Deploy the calculation before running any historical reconciliation. Preserve a before/after export of streak values.

- Feature-flag the reconciliation job and test it against a production-shaped snapshot first.

Primary touchpoints: src/app/services/habit-business.service.ts; src/app/services/habit-interval.service.ts; reset_outdated_habits RPC

05. Make completion undo a complete accounting reversal

P1 financial integrity - Undo removes the owner's completion and earnings but does not debit stockholders, reduce total_dividends_earned, or explicitly reverse dividend distributions. This leaves real balances inconsistent

after a completion with investors.

Implementation plan

- Short-term safety measure: hide/disable undo for any completion that has a dividend_payment. Explain that investor payouts make the event final until the accounting migration is live.

- Replace client-orchestrated undo with a SECURITY DEFINER undo_completion RPC that derives auth.uid(), locks the completion/habit/payment rows, and performs all updates atomically.

- For a dividend-bearing event, insert explicit reversal ledger rows rather than deleting financial history. Debit each recipient's cash and total_dividends_earned by the original distribution, mark the payment reversed, reverse owner earnings, then delete or mark the completion reversed according to the audit policy.

- Define insufficient-funds behavior before implementation. Recommended: do not allow undo if any recipient's available cash is below the reversal amount, unless the system supports a separate receivable/debt ledger.

- Use a durable transaction/ledger model: records should contain completion ID, original/reversal relationship, amount, timestamp, and actor. Avoid relying solely on mutable balance columns for auditability.

Verification

- Complete a dividend-bearing habit, then undo: owner and every holder return to their exact pre-completion balances; holdings and ledger totals reconcile.

- Simulate a recipient spending the dividend before undo; the RPC safely refuses with no partial state.

- Force an error midway through the operation and confirm the transaction rolls back every table change.

Rollback / operational notes

- Release the undo restriction first, then deploy and test the transactional RPC in staging with production-like holdings.

- Reconcile current data before enabling the feature: compare dividend distributions to holder totals and cash movements. Keep reversal records permanently for auditability.

Primary touchpoints: src/app/services/habit-business.service.ts; dividend_payments; stock_dividend_distributions; user_profiles; stock_holdings

Release plan and acceptance checklist

- Phase 0: snapshot the database, document the current schema/function signatures, and restrict unsafe stock RPCs.

- Phase 1: deploy the secure trade RPCs plus client changes; run authorization and concurrency tests in staging.

- Phase 2: deploy multi-goal and scheduling changes behind a feature flag; test daily and specific-day timezones.

- Phase 3: deploy corrected streak rules; validate with seeded completion histories and decide whether to reconcile historical streaks.

- Phase 4: ship the ledger-backed dividend undo flow only after accounting reconciliation, atomic-failure tests, and an operational decision for insufficient funds.

Definition of done

- All money-changing operations are database transactions with authorization derived from auth.uid().

- Automated tests cover the five regression scenarios and concurrent requests.

- Every schema change is a forward-only migration with deployment and rollback notes.

- Production monitoring records failed authorization attempts, rejected completions, RPC errors, and balance-reconciliation mismatches.